

Министерство образования и науки Краснодарского края
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ КРАСНОДАРСКОГО КРАЯ ЕЙСКИЙ
ПОЛИПРОФИЛЬНЫЙ КОЛЛЕДЖ
(ГБПОУ КК ЕПК)

Отделение педагогики

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

ДОКУМЕНТИРОВАНИЕ ИСХОДНОГО КОДА ПРОГРАММ

Автор-разработчик: преподаватель
специальных дисциплин ГБПОУ КК ЕПК
Фомин А. Т.

г. Ейск, 2026 г.

СОДЕРЖАНИЕ

1 Декларативный язык разметки Markdown.....	4
1.1 Базовый синтаксис.....	4
1.1.1 Секционирование документа.....	4
1.1.2 Выделение текста.....	4
1.1.3 Списки.....	4
1.1.4 Таблицы.....	5
1.1.5 Ссылки.....	5
1.1.6 Изображения.....	5
1.1.7 Код.....	5
1.2 Многостраничный документ.....	6
1.2.1 Общие сведения.....	6
1.2.1 Файл README.md.....	10
1.2.2 Файл index.md (раздела).....	10
1.3 Pandoc Markdown.....	12
1.3.1 Общие сведения.....	12
1.3.2 Базовые команды.....	13
2 Система документирования Doxygen.....	15
2.1 Специальные блоки комментариев.....	15
2.2 Блок подробного описания (detailed description).....	15
2.2.1 Первый вариант.....	15
2.2.2 Второй вариант.....	16
2.3 Блок краткого описания (brief description).....	16
2.3.1 Первый вариант.....	17
2.3.2 Второй вариант.....	17
2.4 Однострочные комментарии для элементов.....	17
2.5 Детальное описание метода класса.....	24
2.6 Оформление шапки кода.....	25
2.7 Структурные команды.....	25
2.8 Стилль JavaDoc.....	26
2.9 Разметка Markdown внутри комментариев документации.....	30
2.9.1 Заголовки.....	30
2.9.2 Выделение текста.....	30
2.9.3 Списки.....	31
2.9.4 Ссылки и изображения.....	31
2.9.5 Дополнительные возможности.....	31
2.9.6 Специфические конструкции Doxygen в Markdown.....	32
2.9.7 Таблицы.....	32
2.10 Создание пользовательской страницы.....	34
2.10.1 Использование тега \mainpage в коде.....	34
2.10.2 Создание отдельного Markdown-файла.....	35

2.10.3 Создание отдельной страницы с помощью тега \page.....	36
2.11 Создание нескольких пользовательских страниц.....	36
2.11.1 Использование тега \page в коде.....	36
2.11.2 Создание Markdown-файлов.....	38
2.11.3 Смешанный подход.....	39
2.11.4 Настройка навигации между страницами.....	39
2.12 Настройки Doxygen для вывода в формат PDF.....	40
2.12.1 Поддержка кириллицы и общие настройки.....	40
2.12.2 Компиляция файла LaTeX в PDF-файл.....	41
2.12.3 Настройка титульной страницы и содержания.....	41
2.13 Документирование классов QML.....	43
2.13.1 Установка Doxyqml.....	43
2.13.2 Настройка Doxygen.....	43
2.13.3 Документирование типов. Функции.....	44
2.13.4 Свойства.....	44
2.13.5 Сигналы.....	44
2.13.6 Извлечение внутренних элементов.....	45
2.14 Файл Doxufile и базовые настройки.....	45
2.15 Минимизация документации.....	47
Источники.....	49
Страницы документации.....	49
ПРИЛОЖЕНИЕ А Одностраничная документация в формате MD.....	50

1 Декларативный язык разметки Markdown

1.1 Базовый синтаксис

Markdown — это лёгкий язык разметки для форматирования текста, созданный с целью обозначения форматирования в простом тексте, с максимальным сохранением его читаемости человеком, и пригодный для дальнейшего машинного преобразования. Имеет множество диалектов. В этой связи, ниже представлен лишь базовый синтаксис.

1.1.1 Секционирование документа

Заголовки представлены уровнями от h1 до h6:

```
# Заголовок первого уровня
## Заголовок второго уровня
### Заголовок третьего уровня
```

Первые два имеют альтернативное представление:

```
Заголовок первого уровня
=====
Заголовок второго уровня
-----
```

1.1.2 Выделение текста

```
Курсив — *курсив* или _курсив_
Жирный — **жирный** или __жирный__
Зачёркнутый — ~~зачёркнутый~~ (поддерживается в GitHub Flavored
Markdown, GFM)
```

1.1.3 Списки

Маркированный:

- Пункт 1
- Пункт 2
 - Вложенный пункт

Нумерованный:

1. Первый пункт
2. Второй пункт
3. Третий пункт

1.1.4 Таблицы

Столбец 1	Столбец 2	Столбец 3
Данные 1	Данные 2	Данные 3
Данные 4	Данные 5	Данные 6

1.1.5 Ссылки

Встроенные: [текст ссылки](URL)
С подсказкой: [текст ссылки](URL "подсказка")

Ссылки-сноски:

Текст с [ссылкой][метка]
[метка]: <https://example.com> "Заголовок"

1.1.6 Изображения

![альтернативный текст](путь/к/изображению.png "подпись")

1.1.7 Код

Встроенный: `код`

Блок кода (с указанием языка для подсветки синтаксиса):

```

```cpp
#include <iostream>
#include <vector>

int main() {
 std::vector<int> numbers = {1, 2, 3, 4, 5};

 for (const auto& num : numbers) {
 std::cout << num << " ";
 }
 std::cout << std::endl;

 return 0;
}
```

```

1.2 Многостраничный документ

1.2.1 Общие сведения

Пример одностраничной документации в формате Markdown показан в Приложении А [50].

Для больших проектов документация разделяется на несколько файлов.

Типичная структура проекта (без определенных разделов):

```

проект/
├── README.md
├── index.md (главная страница)
├── 01-introduction.md
├── 02-features.md
├── 03-settings.md
└── appendix/
    ├── glossary.md
    └── contacts.md

```

В таком проекте главным файлом является index.md или mainpage.md.

Структура главного файла многостраничной документации:

1. Заголовок (h1)

- название проекта/документации крупным шрифтом;

- бейджи статуса, лицензии, версии (опционально).
- 2. Краткое описание (1–3 абзаца)**
- назначение проекта;
 - решаемые задачи;
 - целевая аудитория;
 - ключевые возможности.
- 3. Навигация по разделам**
- структурированный список основных разделов документации с кратким описанием каждого;
 - используйте нумерованные или маркированные списки со ссылками на подразделы.
- 4. Быстрый старт (Quick Start)**
- краткая пошаговая инструкция для самых распространённых сценариев использования;
 - 3–5 шагов для быстрого начала работы.
- 5. Структура документации**
- визуальная схема или список разделов с указанием, где найти нужную информацию;
 - например: «Для установки см. раздел „Установка“, для API — раздел „Документация API“».
- 6. Основные концепции**
- краткий глоссарий ключевых терминов проекта;
 - описание архитектуры или принципов работы (если применимо).
- 7. Ссылки на важные разделы**
- «Начало работы»;
 - «Руководство пользователя»;
 - «API-документация»;
 - «Настройка и конфигурация»;
 - «Устранение неполадок»;
 - «Вклад в проект» (для открытых проектов).
- 8. Визуальные элементы (опционально)**
- схема архитектуры;
 - скриншот интерфейса;
 - GIF-анимация работы функционала.
- 9. Контакты и поддержка**
- способы связи с разработчиками;

- ссылки на чаты сообщества (Discord, Slack);
- ссылка на систему отслеживания ошибок (GitHub Issues).

10. Лицензия

- указание лицензии проекта;
- ссылка на полный текст лицензии (LICENSE).

Пример docs/index.md для проекта «База данных метеоритов»

```
# Документация MeteoriteDB

![License](https://img.shields.io/badge/License-MIT-green)
![Build](https://img.shields.io/badge/Build-passing-brightgreen)

Официальная документация настольного приложения для учёта и
визуализации данных о метеоритах.

## Краткое описание

**MeteoriteDB** – приложение на Qt6 для:
* учёта записей о метеоритах;
* визуализации данных в таблицах и графиках;
* импорта/экспорта данных (CSV/JSON);
* настройки базы данных и логирования.

## Навигация по документации

### Основные разделы

1. **[Начало работы](getting-started.md)**
   * требования;
   * установка и сборка;
   * первый запуск.

2. **[Руководство пользователя](user-guide.md)**
   * интерфейс приложения;
   * добавление и редактирование записей;
   * импорт и экспорт данных;
   * работа с графиками.

3. **[Настройка и конфигурация](configuration.md)**
   * файл `settings.ini`;
```


- * структура базы данных;
- * настройка логирования.

4. **[Устранение неполадок](troubleshooting.md)**

- * частые проблемы и их решения;
- * диагностика ошибок.

5. **[Разработка](development.md)**

- * архитектура проекта;
- * вклад в проект;
- * стандарты кодирования.

Быстрый старт

Чтобы начать работу:

1. Перейдите в раздел **[Начало работы](getting-started.md)**.
2. Установите зависимости и соберите проект.
3. Запустите приложение и создайте базу данных через вкладку **«Настройки»**.
4. Заполните данные о студенте на вкладке **«Информация»**.
5. Добавляйте записи о метеоритах на вкладке **«Редактирование»**.

Структура проекта

Документация организована по принципу «от простого к сложному»:

- * разделы для конечных пользователей — в начале;
- * технические разделы — в конце.

Контакты

- * **Автор:** [Ваше имя]
- * **Email:** [ваш email]
- * **Репозиторий:** [ссылка]
- * **Чат сообщества:** [Discord-ссылка]

Лицензия

Документация распространяется под лицензией [Creative Commons Attribution4.0](LICENSE-docs). Проект — под лицензией [MIT](LICENSE).

1.2.1 Файл README.md

README.md — это текстовый файл в формате Markdown, который размещается в корневой директории проекта и содержит основную информацию о проекте. Если в проекте нет других файлов пользовательской документации, то именно в этом файле содержится подробная информация о всем проекте, в противном случае информация в README.md подается в краткой форме.

Типичный файл включает следующие разделы:

1. **Название проекта** (заголовок h1) — кратко и ясно отражает суть.
2. **Краткое описание** — 1–2 предложения о назначении и решаемых задачах.
3. **Особенности и преимущества** — ключевые функции и выгоды.
4. **Установка** — пошаговая инструкция по развёртыванию:
 - клонирование репозитория;
 - установка зависимостей;
 - настройка окружения.
5. **Использование** — примеры запуска и применения:
 - команды CLI;
 - фрагменты кода;
 - скриншоты интерфейса.
6. **Требования** — версии ПО, библиотек, ОС.
7. **Вклад в проект** (Contributing) — правила отправки pull request'ов.
8. **Лицензия** — тип лицензии (MIT, Apache, GPL и т. д.).
9. **Контакты** — email, соцсети, каналы связи.
10. **Благодарности** (Acknowledgements) — ссылки на сторонние библиотеки, инструменты.

1.2.2 Файл index.md (раздела)

Если проект многостраничной документации содержит (в отдельных директориях) тематические разделы документации, то каждый такой раздел имеет свой файл index.md — главную страницу раздела.

Структура такого проекта может выглядеть так:

```
docs/
├── index.md          # Главная страница всей документации
├── installation/
│   ├── index.md      # Главная страница раздела «Установка»
│   ├── windows.md    # Установка на Windows
│   └── linux.md       # Установка на Linux
└── usage/
    ├── index.md       # Главная страница раздела «Использование»
    ├── basics.md      # Основы работы
    └── advanced.md    # Продвинутые возможности
```

Обычно файл включает следующие элементы:

1. **Заголовок раздела (H1)** — название раздела.
2. **Краткое введение** (1–3 абзаца):
 - назначение раздела;
 - для кого он предназначен;
 - какие вопросы решает.
3. **Оглавление раздела** — нумерованный или маркированный список ссылок на ключевые страницы раздела.
4. **Ключевые концепции** — краткий обзор основных идей, терминов или принципов раздела.
5. **Быстрые ссылки** — ссылки на самые востребованные страницы (например, «Начало работы», «Частые вопросы», «Устранение неполадок»).
6. **Визуальные элементы** (опционально):
 - схемы;
 - диаграммы;
 - скриншоты.
7. **Ссылки на смежные разделы** — указания, куда перейти за дополнительной информацией.

Таблица 1 — Отличие index.md от README.md

| Параметр | README.md | index.md |
|---------------------|--|----------------------------------|
| Расположение | Корень репозитория | В директории раздела |
| Назначение | Общее описание проекта | Навигация по конкретному разделу |
| Аудитория | Все пользователи (включая разработчиков) | Пользователи конкретного раздела |

| Параметр | README.md | index.md |
|------------|--|--|
| Содержание | Обзор проекта, установка, использование, вклад | Введение в раздел + ссылки на страницы раздела |

1.3 Pandoc Markdown

1.3.1 Общие сведения

Pandoc Markdown — расширенная версия Markdown, используемая в инструменте Pandoc. Pandoc — это универсальный конвертер документов с открытым исходным кодом, написанный на языке Haskell. Он позволяет конвертировать текст между множеством форматов разметки и обработки документов.

Pandoc поддерживает конвертацию между следующими форматами:

- различные варианты Markdown;
- HTML;
- LaTeX;
- Microsoft Word (.docx);
- ODT (OpenDocument Text);
- PDF (через LaTeX, ConTeXt или HTML);
- EPUB;
- RTF;
- и многие другие.

Pandoc расширяет стандартный Markdown дополнительными синтаксическими конструкциями:

- **таблицы** (в разных форматах: простой, с выравниванием, многострочные);
- **списки определений** (term :: определение);
- **сноски** ([^1] и [^1]: текст сноски);
- **цитаты и библиография** (поддержка CSL — Citation Style Language);
- **математические формулы** (через LaTeX-синтаксис $...$ и $...$);
- **метаданные документа** (YAML-заголовки);
- **перекрёстные ссылки**;
- **встроенные комментарии**;
- **блоки кода** с указанием языка и подсветкой синтаксиса;
- **диаграммы** (Mermaid, Graphviz и др.).

1.3.2 Базовые команды

Конвертация Markdown в другие форматы:

```
# Markdown → HTML
pandoc input.md -o output.html

# Markdown → PDF (через LaTeX)
pandoc input.md -o output.pdf

# Markdown → Word
pandoc input.md -o output.docx

# Markdown → LaTeX
pandoc input.md -o output.tex
```

Конвертация из других форматов в Markdown:

```
# HTML → Markdown
pandoc input.html -o output.md

# Word → Markdown
pandoc input.docx -o output.md
```

Объединение нескольких файлов в один документ:

```
pandoc file1.md file2.md -o merged_output.docx
```

Пример документа с метаданными

```
---
title: "Документация MeteoriteDB"
author: "Иван Иванов"
date: "2024-05-20"
keywords: [метеориты, база данных, Qt6]
---

# Введение

Это пример документа в формате Pandoc Markdown.
```

Математическая формула

Уравнение Эйнштейна: $E = mc^2$

Таблица

| Название | Масса (кг) | Год обнаружения |
|-------------|------------|-----------------|
| ----- | ----- | ----- |
| Челябинский | 11000 | 2013 |
| Тунгусский | ~100000000 | 1908 |

[^1]: Дополнительная информация о метеорите.

2 Система документирования Doxygen

2.1 Специальные блоки комментариев

Специальный блок комментариев - это блок комментариев в стиле С или С++ с некоторыми дополнительными пометками (тегами), которые воспринимаются Doxygen, как фрагмент структурированного текста, который должен быть включен в создаваемую документацию.

Для каждой сущности в коде существует два (или в некоторых случаях три) типа описаний, которые вместе образуют документацию для этой сущности: краткое описание (brief description) и подробное описание (detailed description). Оба не являются обязательными.

Для методов и функций существует также третий тип описания, так называемое описание в теле ("in body" description), которое состоит из объединения всех блоков комментариев, найденных в теле метода или функции.

Допускается наличие более одного краткого или подробного описания (но не рекомендуется, поскольку порядок, в котором описания будут отображаться, не указан).

Как следует из названия, краткое описание - это короткая однострочная информация, в то время как подробное описание содержит более длинную и подробную документацию. Есть несколько способов создания блоков подробного описания.

2.2 Блок подробного описания (detailed description)

2.2.1 Первый вариант

Вариант оформления в стиле Qt (предпочтительный) — к комментариям в стиле С добавляется восклицательный знак. Обратите внимание, что среда программирования Qt Creator автоматизирует процесс документирования кода.

```
/*!  
 * ... текст ...  
*/
```

Промежуточные звездочки не обязательны:

```
/*!  
... текст ...  
*/
```

2.2.2 Второй вариант

Основан на использовании блока, состоящего, как минимум, из двух строк комментариев в стиле C++, где каждая строка начинается с дополнительной косой черты или восклицательного знака. Например:

```
///  
/// ... текст ...  
///
```

Или

```
///  
///  
///  
///!
```

2.3 Блок краткого описания (brief description)

Для краткого описания также есть несколько возможностей. Можно использовать команду `\brief` с одним из приведенных выше блоков комментариев. Эта команда заканчивается в конце абзаца, поэтому подробное описание следует после пустой строки.

```
/*! \brief Краткое описание.  
* Продолжение краткого описания.  
*  
* Здесь начинается подробное описание.  
*/
```


Возможны также несколько вариантов без использования команды \brief, но в сочетании блоков brief description и detailed description.

2.3.1 Первый вариант

```
/// Краткое описание.  
/** Подробное описание. */
```

Или

```
//! Краткое описание.  
/** Подробное описание. */
```

2.3.2 Второй вариант

```
//! Краткое описание.  
  
//! Подробное описание  
//! начинается здесь.
```

Обратите внимание на пустую строку в последнем примере, которая требуется для отделения краткого описания от блока, содержащего подробное описание. В этом случае для параметра JAVADOC_AUTOBRIEF:

```
JAVADOC_AUTOBRIEF = NO
```

2.4 Однострочные комментарии для элементов

Если нужно задокументировать элементы файла, структуры, объединения, класса или перечисления, необходимо поместить блок документации после элемента, а не перед ним. Для этого вам необходимо поместить дополнительный маркер < в блок комментариев. Обратите внимание, что аналогичный подход применяется и для параметров функции.

Например, в стиле Qt:

```
int var; /*!< Подробное описание после элемента */
```

Или

```
int var; /**< Подробное описание после элемента */
```

Но чаще всего требуется лишь указать краткое описание

```
int var; //!< Краткое описание после элемента
```

Или

```
int var; ///< Краткое описание после элемента
```

Листинг 1. Пример документирования класса

```
1.  /*!
2.   * \file piechartwidget.h
3.   * \brief Заголовочный файл виджета круговой диаграммы на
      основе Qt Charts.
4.   *
5.   * Файл содержит объявление класса PieChartWidget – виджета
      для отображения
6.   * круговой диаграммы с данными из SQLite-базы данных.
7.   */
8.
9.  #ifndef PIECHARTWIDGET_H
10. #define PIECHARTWIDGET_H
11.
12. #include <QWidget>
13. #include <QtCharts>
14. #include <QMap>
15. #include <QString>
16.
17. /*!
18.  * \class PieChartWidget
```

```

19.  * \brief Виджет для отображения круговой диаграммы с данными
    из базы данных.
20.  *
21.  * Виджет использует Qt Charts для визуализации данных в виде
    круговой диаграммы.
22.  * Поддерживает загрузку данных из SQLite-базы данных и
    автоматическое обновление
23.  * графика. Отображает легенду и подписи сегментов.
24.  *
25.  * \note Для работы требуется модуль Qt6::Charts.
26.  */
27.  class PieChartWidget : public QWidget
28.  {
29.      Q_OBJECT
30.
31.  public:
32.      /*!
33.       * \brief Конструктор виджета круговой диаграммы.
34.       *
35.       * Создаёт виджет с инициализированным графиком, но без
    данных.
36.       * График настраивается с анимацией, антиалиасингом и
    видимой легендой.
37.       *
38.       * \param parent Родительский виджет (по умолчанию –
    nullptr).
39.       */
40.      explicit PieChartWidget(QWidget *parent = nullptr);
41.
42.      /*!
43.       * \brief Загружает данные из SQLite-базы данных и строит

```

круговую диаграмму.

```
44.      *
45.      * Выполняет следующие действия:
46.      * - подключается к указанной SQLite-базе данных;
47.      * - выполняет SQL-запрос для получения данных из
      заданной таблицы;
48.      * - очищает предыдущие данные на диаграмме;
49.      * - добавляет новые сегменты на круговую диаграмму;
50.      * - отображает подписи сегментов в формате «Категория:
      Значение».
51.      *
52.      * \param dbPath Путь к файлу SQLite-базы данных.
53.      * \param table Название таблицы, из которой берутся
      данные.
54.      * \param categoryField Название поля таблицы,
      содержащего категории (метки сегментов).
55.      * \param valueField Название поля таблицы, содержащего
      числовые значения для сегментов.
56.      *
57.      * \note Если подключение к БД или выполнение запроса
      завершается ошибкой, выводится сообщение в qDebug().
58.      * Данные автоматически обновляются при повторном вызове
      метода.
59.      */
60.      void loadDataFromDatabase(const QString &dbPath,
61.                                const QString &table,
62.                                const QString &categoryField,
63.                                const QString &valueField);
64.
65. private:
```

```

66.      /*!
67.      * \brief Указатель на виджет отображения графика.
68.      *
69.      * Содержит экземпляр QChartView, который отображает
      QChart с круговой диаграммой.
70.      * Настраивается в конструкторе: включается антиалиасинг
      для сглаживания линий.
71.      */
72.      QChartView *m_chartView;
73.
74.      /*!
75.      * \brief Серия данных для круговой диаграммы.
76.      *
77.      * Хранит данные в виде сегментов (QPieSlice), которые
      отображаются на графике.
78.      * Очищается перед загрузкой новых данных в методе
      loadDataFromDatabase().
79.      */
80.      QPieSeries *m_series;
81.  };
82.
83.  #endif // PIECHARTWIDGET_H

```

Листинг 2. Более общий пример документирования класса

```

1.  //!< Класс тестирования.
2.  /*!
3.      Более подробное описание класса.
4.  */
5.
6.  class QTeststyle_Test
7.  {

```

```

8.  public:
9.
10.     ///  
11.     ///  
12.     enum TEnum {  
13.         TVal1, ///  
14.         TVal2, ///  
15.         TVal3 ///  
16.     }  
17.     ///  
18.     ///  
19.     *enumPtr,  
20.     ///  
21.     ///  
22.     enumVar;  
23.  
24.     ///  
25.     ///  
26.         ///  
27.     ///  
28.     QTstyle_Test();  
29.  
30.     ///  
31.     ///  
32.         ///  
33.     ///  
34.     ~QTstyle_Test();  
35.  
36.     ///  


```

```

37.      /*!
38.          \param a целочисленный аргумент.
39.          \param s константный указатель на символьную строку.
40.          \return Результаты тестирования
41.          \sa QTeststyle_Test(), ~QTeststyle_Test(), testMeToo() and
      publicVar()
42.      */
43.      int testMe(int a, const char *s);
44.
45.      //! Чисто виртуальный метод.
46.      /*!
47.          \sa testMe()
48.          \param c1 первый аргумент.
49.          \param c2 второй аргумент.
50.      */
51.      virtual void testMeToo(char c1, char c2) = 0;
52.
53.      //! Общедоступная переменная.
54.      /*!
55.          Описание.
56.      */
57.      int publicVar;
58.
59.      //! Указатель на функцию.
60.      /*!
61.          Описание.
62.      */
63.      int (*handler)(int a, int b);
64.  };

```

2.5 Детальное описание метода класса

Для подробного описания методов могут применяться следующие специальные команды:

\param - Параметры передаваемые функции. Пример:

```
\param name Имя пользователя
```

Может применяться с опциональным атрибутом «направление». Этот атрибут может иметь три значения: [in], [out], [in, out]. Например:

```
/*!  
    Копирует содержимое из исходной области памяти в целевую  
    область памяти  
    \param[out] dest Целевая область памяти  
    \param[in] src Исходная область памяти  
    \param[in] n Количество байтов, которые необходимо  
    скопировать  
*/  
void memcpy(void *dest, const void *src, size_t n);
```

\return - Возвращаемое функцией значение. Пример:

```
\return Информация о пользователе
```

\throw - Исключения выбрасываемые функцией. Пример:

```
\throw DatabaseError Если произошла ошибка при подключении к  
базе данных
```

\sa - Начинается абзац, в котором может быть указана одна или несколько перекрестных ссылок на классы, функции, методы, переменные, файлы или URL.

2.6 Оформление шапки кода

Главный файл программы обычно содержит «шапку» с подробным описанием проекта (не обязательно, если эта информация документируется в отдельном файле).

```
/*!  
 * \brief Pretty nice class.  
 * \details This class is used to demonstrate a number of  
 * section commands.  
 * \author John Doe  
 * \author Jan Doe  
 * \version 4.1a  
 * \date 1990-2011  
 * \pre First initialize the system.  
 * \bug Not all memory is freed when deleting an object of this  
 * class.  
 * \warning Improper use can crash your application  
 * \copyright GNU Public License.  
 */  
class SomeNiceClass {};
```

2.7 Структурные команды

Иногда удобнее размещать документацию в другом месте. Doxygen позволяет размещать блоки документации практически где угодно (за исключением тела функции или обычного блока комментариев в стиле C).

1. Блок документации, который не размещается непосредственно перед (или после) элемента, содержит структурные команды внутри блока документации. Это может привести к дублированию информации. Поэтому на практике следует избегать использования структурных команд, если только этого не требуют исключительные обстоятельства.
2. Структурные команды начинаются с обратной косой черты (\). Далее следует имя команды и один или несколько параметров.

К структурным командам относятся:

- **\struct** для документирования C-структуры.
- **\union** для документирования объединения.

- **\enum** для документирования типа перечисления.
- **\fn** для документирования функции.
- **\var** для документирования переменной, **typedef** или значения перечисления.
- **\def** для документирования **#define**.
- **\typedef** для документирования определения типа.
- **\file** для документирования файла.
- **\namespace** для документирования пространства имён.
- **\package** для документирования пакета Java.
- **\interface** для документирования интерфейса IDL.

Применять структурные команды — не рекомендуется!

Подробные сведения и примеры оформления структурных команд см. в документации: <https://www.doxygen.nl/manual/docblocks.html>

2.8 Стилль JavaDoc

JavaDoc-комментарии начинаются с **/**** и заканчиваются ***/**. Они должны располагаться перед документируемым элементом (классом, методом, полем и т.д.).

Листинг 3. Пример синтаксиса

```

1.  /**
2.   * @file piechartwidget.h
3.   * @brief Заголовочный файл виджета круговой диаграммы на
      основе Qt Charts.
4.   *
5.   * Файл содержит объявление класса PieChartWidget – виджета
      для отображения
6.   * круговой диаграммы с данными из SQLite-базы данных.
7.   */
8.
9.  #ifndef PIECHARTWIDGET_H
10. #define PIECHARTWIDGET_H
11.

```

```

12. #include <QWidget>
13. #include <QtCharts>
14. #include <QMap>
15. #include <QString>
16.
17. /**
18.  * @class PieChartWidget
19.  * @brief Виджет для отображения круговой диаграммы с данными
      из базы данных.
20.  *
21.  * Виджет использует Qt Charts для визуализации данных в виде
      круговой диаграммы.
22.  * Поддерживает загрузку данных из SQLite-базы данных и
      автоматическое обновление
23.  * графика. Отображает легенду и подписи сегментов.
24.  *
25.  * @note Для работы требуется модуль Qt6::Charts.
26.  */
27. class PieChartWidget : public QWidget
28. {
29.     Q_OBJECT
30.
31. public:
32.     /**
33.      * @brief Конструктор виджета круговой диаграммы.
34.      *
35.      * Создаёт виджет с инициализированным графиком, но без
      данных.
36.      * График настраивается с анимацией, антиалиасингом и
      видимой легендой.

```

```

37.      *
38.      * @param parent Родительский виджет (по умолчанию –
      nullptr).
39.      */
40.      explicit PieChartWidget(QWidget *parent = nullptr);
41.
42.      /**
43.      * @brief Загружает данные из SQLite-базы данных и строит
      круговую диаграмму.
44.      *
45.      * Выполняет следующие действия:
46.      * – подключается к указанной SQLite-базе данных;
47.      * – выполняет SQL-запрос для получения данных из
      заданной таблицы;
48.      * – очищает предыдущие данные на диаграмме;
49.      * – добавляет новые сегменты на круговую диаграмму;
50.      * – отображает подписи сегментов в формате «Категория:
      Значение».
51.      *
52.      * @param dbPath Путь к файлу SQLite-базы данных.
53.      * @param table Название таблицы, из которой берутся
      данные.
54.      * @param categoryField Название поля таблицы,
      содержащего категории (метки сегментов).
55.      * @param valueField Название поля таблицы, содержащего
      числовые значения для сегментов.
56.      *
57.      * @note Если подключение к БД или выполнение запроса
      завершается ошибкой, выводится сообщение в qDebug().
58.      * Данные автоматически обновляются при повторном вызове
      метода.

```

```

59.      */
60.      void loadDataFromDatabase(const QString &dbPath,
61.                                const QString &table,
62.                                const QString &categoryField,
63.                                const QString &valueField);
64.
65. private:
66.     /**
67.      * @brief Указатель на виджет отображения графика.
68.      *
69.      * Содержит экземпляр QChartView, который отображает
70.      * QChart с круговой диаграммой.
71.      * Настраивается в конструкторе: включается антиалиасинг
72.      * для сглаживания линий.
73.      */
74.     QChartView *m_chartView;
75.
76.     /**
77.      * @brief Серия данных для круговой диаграммы.
78.      *
79.      * Хранит данные в виде сегментов (QPieSlice), которые
80.      * отображаются на графике.
81.      * Очищается перед загрузкой новых данных в методе
82.      * loadDataFromDatabase().
83.      */
84.     QPieSeries *m_series;
85. };
86.
87. #endif // PIECHARTWIDGET_H

```

Чтобы Doxygen корректно обрабатывал JavaDoc-комментарии, необходимо в файле Doxyfile установить параметр:

```
JAVADOC_AUTOBRIEF = YES
```

2.9 Разметка Markdown внутри комментариев документации

Doxygen поддерживает подмножество синтаксиса Markdown — не все конструкции будут работать. Чтобы включить поддержку Markdown, убедитесь, что в конфигурационном файле Doxyfile задано:

```
MARKDOWN_SUPPORT = YES
```

2.9.1 Заголовки

Можно использовать синтаксис с решетками:

```
/// # Заголовок первого уровня
/// ## Заголовок второго уровня
/// ### Заголовок третьего уровня
/**
 * # Основной заголовок модуля
 * ## Описание функций
 */
```

2.9.2 Выделение текста

- **Жирный текст:** ****жирный**** или **__жирный__**
- *Курсив:* **курсив** или *_курсив_*
- ~~Зачёркнутый:~~ ~~~~зачёркнутый~~~~

Пример:

```
/// Функция **обязательно** должна быть вызвана *до*
/// инициализации.
/// ~~Устаревший метод~~ — не использовать.
```

2.9.3 Списки

Нумерованные:

```
/// 1. Первый шаг
/// 2. Второй шаг
/// 3. Третий шаг
```

Маркированные:

```
/// - Пункт 1
/// - Пункт 2
///   - Вложенный пункт
/// - Пункт 3
```

2.9.4 Ссылки и изображения

Ссылки

- Встроенные: [текст ссылки](URL)
- С подписью: [ссылка][метка] и далее [метка]: URL "Подсказка"

Пример:

```
/// Посетите [сайт проекта](https://example.com) для получения
дополнительной информации.
```

Изображения

```
/// ![Альтернативный текст] (путь/к/изображению.png "Подпись")
```

2.9.5 Дополнительные возможности

Код

- Встроенный код: `код`
- Блоки кода с указанием языка:

```
/// ```cpp
/// int main() {
///     return 0;
```

```
/// }  
/// ```
```

Цитаты

```
/// > Это цитата из документации.  
/// > Она может занимать несколько строк.
```

Горизонтальная линия

```
/// ***  
/// ---  
/// ---
```

2.9.6 Специфические конструкции Doxygen в Markdown

Можно комбинировать Markdown с командами Doxygen:

```
/**  
 * ## Пример функции  
 *  
 * Эта функция выполняет важные операции.  
 *  
 * @param value Входной параметр  
 * @return Результат вычислений  
 * @see другаяФункция()  
 *  
 * **Пример использования:**  
 * ```cpp  
 * int result = myFunction(42);  
 * ```  
 */  
int myFunction(int value);
```

2.9.7 Таблицы

Поддерживается синтаксис таблиц Markdown:


```

/// | Параметр | Тип | Описание |
/// |-----|-----|-----|
/// | `value` | `int` | Входной параметр |
/// | `flag` | `bool` | Флаг активации |

```

Пример комплексной документации.

```

/**
 * # Класс Calculator
 *
 * Мощный калькулятор для выполнения арифметических операций.
 *
 * ## Особенности
 * - Поддержка сложения и умножения
 * - Высокая точность вычислений
 * - ~~Деление (в разработке)~~
 *
 * ## Использование
 *
 * ```cpp
 * Calculator calc;
 * double result = calc.add(5.5, 3.2); // 8.7
 * ```
 *
 * | Метод | Описание |
 * |-----|-----|
 * | `add()` | Сложение двух чисел |
 * | `multiply()` | Умножение двух чисел |
 *
 * @author Иванов И. И.
 * @version 1.0
 * @since 2023-10-01
 */
class Calculator {
public:
    /**
     * Складывает два числа.

```

```

*
* > Важно: числа должны быть в допустимом диапазоне.
*
* @param a Первое слагаемое
* @param b Второе слагаемое
* @return Сумма `a` и `b`
* @see multiply()
*/
double add(double a, double b);
};

```

2.10 Создание пользовательской страницы

2.10.1 Использование тега \mainpage в коде

Самый простой способ — добавить специальный комментарий с тегом \mainpage в один из исходных файлов проекта.

```

/*!
* \mainpage 0 программе
*
* Это главная страница документации, она же — страница «0
программе».
*
* \section intro_sec Введение
* Наше приложение предназначено для решения задач...
*
* \subsection features_subsec Основные возможности
* – Функция 1: описание
* – Функция 2: описание
* – Функция 3: описание
*
* \section version_sec Версия
* Текущая версия: 1.0.0
*
* \section contact_sec Контакты

```

```
* Для обратной связи: developer@example.com
*/
```

2.10.2 Создание отдельного Markdown-файла

Если документация создается отдельно от кода в виде MD-файла, то необходимо создать файл `about.md` (или `mainpage.md`, или `index.md`) со следующим содержимым:

```
# 0 программе {#mainpage}

Это главная страница документации.

## Введение

Наше приложение предназначено для решения задач...

## Основные возможности

- **Функция 1**: описание функциональности
- **Функция 2**: описание функциональности
- **Функция 3**: описание функциональности

## Версия

Текущая версия: 1.0.0

## Контакты

Для обратной связи: developer@example.com
```

В файле конфигурации Doxygen (Doxyfile) необходимо установить следующие параметры:

```
INPUT += path/to/about.md
USE_MDFILE_AS_MAINPAGE = YES
FILE_PATTERNS += *.md *.markdown
```

2.10.3 Создание отдельной страницы с помощью тега \page

Если «О программе» должна быть не главной страницей, а отдельной страницей в документации, то необходимо добавить в код (или в отдельный .dox-файл) следующий комментарий:

```
/*!  
 * \page aboutpage 0 программе  
 *  
 * \section about_intro Введение  
 * Описание программы...  
 *  
 * \section about_features Возможности  
 * – Возможность 1  
 * – Возможность 2  
 *  
 * \section about_version Версия  
 * Версия 1.0.0  
 */
```

Чтобы добавить ссылку на эту страницу в главное меню, в любом другом комментарии используйте:

```
/*!  
 * \mainpage Главная страница  
 * Добро пожаловать в документацию!  
 *  
 * Подробнее о программе: \ref aboutpage  
 */
```

2.11 Создание нескольких пользовательских страниц

2.11.1 Использование тега \page в коде

Для каждой страницы добавьте блок комментариев с тегом \page.

Пример для «about»

```
/*!  
 * \page about 0 программе  
 *  
 * \section about_intro Введение  
 * Это наше приложение для...  
 *  
 * \section about_features Возможности  
 * - Функция 1  
 * - Функция 2  
 */
```

Пример для «installation»:

```
/*!  
 * \page installation Инструкция по установке  
 *  
 * \section install_req Требования  
 * - ОС: Linux/Windows/macOS  
 * - Память: 2GB RAM  
 *  
 * \section install_steps Шаги установки  
 * 1. Скачайте архив  
 * 2. Распакуйте  
 * 3. Запустите install.sh  
 */
```

Пример для «usage»

```
/*!  
 * \page usage Руководство пользователя  
 *  
 * \section usage_start Начало работы  
 * После установки запустите:  
 * \code  
 * ./myapp --help  
 * \endcode  
 */
```

2.11.2 Создание Markdown-файлов

Создайте отдельные .md-файлы для каждой страницы.
about.md:

```
# 0 программе {#about}

## Введение

Это наше приложение для...

## Возможности

- Функция 1: описание
- Функция 2: описание
```

installation.md:

```
# Инструкция по установке {#installation}

## Требования

- ОС: Linux/Windows/macOS
- Память: 2GB RAM

## Шаги установки

1. Скачайте архив
2. Распакуйте
3. Запустите `install.sh`
```

В Doxyfile настройте обработку Markdown:

```
MARKDOWN_SUPPORT = YES
INPUT += docs/about.md docs/installation.md docs/usage.md
FILE_PATTERNS += *.md *.markdown
```

2.11.3 Смешанный подход

Используйте \page для основной структуры и Markdown для контента.
В коде:

```
/*! \page about 0 программе */  
/*! \page installation Установка */  
/*! \page usage Использование */
```

В Markdown-файлах:

- page_about.md – контент для страницы «0 программе»;
- page_installation.md – инструкция по установке;
- page_usage.md – руководство пользователя.

В Doxyfile:

```
INPUT += page_about.md page_installation.md page_usage.md
```

2.11.4 Настройка навигации между страницами

Чтобы добавить ссылки между страницами, используйте тег \ref.
В любой странице:

```
/*!  
* Смотрите также:  
* - \ref about "0 программе"  
* - \ref installation "Инструкция по установке"  
* - \ref usage "Руководство пользователя"  
*/
```

В Markdown:

Смотрите также: [0 программе](@ref about), [Установка](@ref installation)

Или через HTML-ссылки в Markdown:

Смотрите также: 0 программе

2.12 Настройки Doxygen для вывода в формат PDF

2.12.1 Поддержка кириллицы и общие настройки

Для получения файла документации в формате PDF, с поддержкой кириллицы, в дистрибутиве Fedora Linux необходимо установить следующие пакеты:

```
$ sudo dnf install doxygen-latex texlive-xetex texlive-fontspec  
texlive-babel-russian texlive-polyglossia texlive-cyrillic  
texlive-collection-langcyrillic texlive-luatex texlive-unicode-  
data
```

В Doxyfile должны быть установлены следующие параметры:

```
GENERATE_LATEX = YES  
LATEX_OUTPUT = latex  
LATEX_CMD_NAME = pdflatex  
MAKEINDEX_CMD_NAME = makeindex  
PDF_HYPERLINKS = YES  
USE_PDFLATEX = YES  
EXTRA_PACKAGES = graphicx, hyperref, color, fancyhdr
```

Примечание. EXTRA_PACKAGES задаёт список LaTeX-пакетов, которые нужно подключить в преамбуле документа (в секции `\usepackage{}`). Это расширяет базовые возможности LaTeX для форматирования, вёрстки и оформления.

- **graphicx** — работа с графикой. Позволяет вставлять изображения (PNG, JPG, PDF, SVG и др.), масштабировать, обрезать, поворачивать их. Ключевая команда: `\includegraphics[options]{filename}`.
- **hyperref** — назначение: создает гиперссылки и закладки в PDF, что позволяет делать кликабельные ссылки на разделы, рисунки, таблицы, библиографию.
- **color** — (или современный аналог `xcolor`); назначение: управление цветами в документе, что позволяет задавать цвета текста (`\textcolor{red}{важное замечание}`);
- **fancyhdr** — назначение: гибкое оформление верхних и нижних колонтитулов (headers и footers).

2.12.2 Компиляция файла LaTeX в PDF-файл

Для сборки набора исходников TeX можно использовать любой TeX-редактор с возможностью компиляции исходного кода TeX в различные форматы (PS, PDF, DVI).

Рекомендуется использовать приложение TeXStudio. Данная программа входит в пакетную базу дистрибутива Fedora:

```
$ sudo dnf install texstudio
```

- Запустите программу и откройте в редакторе файл `refman.tex`, находящийся в директории `/latex` после генерации документации в формат LaTeX.
- Запустите сборку Инструменты → Компиляция (или F6).

2.12.3 Настройка титульной страницы и содержания

После сборки документации в формат Latex, необходимо вручную внести изменения в файл `refman.tex` для вывода своей титульной страницы и редактирования содержания.

Измените поля документа:

```
\usepackage{geometry}
\geometry{
  a4paper,
  top=2cm,
  bottom=2cm,
  left=2cm,
  right=2cm
}
```

Перед секцией `\begin{document}` установите команду отключающую вывод всех пустых страниц в документе:

```
% Переопределение команды для устранения пустых страниц
\let\cleardoublepage\clearpage
\begin{document}
```

Измените код между командами `\begin{titlepage}` и `\tableofcontents`:

```

\begin{titlepage}
  \begin{center}
    % Верхняя часть: организация
    {\normalsize Министерство образования и науки
Краснодарского края}\\
    {\bfseries\small ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ПРОФЕССИОНАЛЬНОЕ\\
    ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ КРАСНОДАРСКОГО КРАЯ\\
    "<ЕЙСКИЙ ПОЛИПРОФИЛЬНЫЙ КОЛЛЕДЖ">\\
    (ГБПОУ КК ЕПК)}\\
    \rule{\textwidth}{0.4pt} \\\[0.5cm] %
разделительная линия
    \vspace{\stretch{2}}
    % Логотип по центру
    \includegraphics[width=4cm]{../pic/logo.png}
    \vspace{\stretch{1.5}}
    % Название проекта
    {\huge\bfseries Программа myDia}
    \vspace{\stretch{1}}

    % Версия
    {\large Версия 1.0}
    \vspace{\stretch{2}}

    % Автор
    {\Large Автор: [Ваше имя/группа]}
    \vspace{\stretch{3}} % Больше пространства внизу

    % Нижняя часть: город и год
    {\normalsize г. Ейск, 2026 год}

  \end{center}
\end{titlepage}
% re-enable anchors again
\hypersetup{pageanchor=true}
%--- Begin generated contents ---
\clearpage

```

```
\pagenumbering{roman}
\thispagestyle{plain}
\begin{center}
  \vspace*{1cm}
  {\LARGE\bfseries СОДЕРЖАНИЕ}
\end{center}
\vspace{0.5cm} % отступ после заголовка

\tableofcontents
```

2.13 Документирование классов QML

2.13.1 Установка Doxyqml

Файлы QML можно документировать в системе Doxygen с помощью фильтра Doxyqml. Он интегрируется как фильтр ввода Doxygen, превращая файлы .qml в псевдо-C++, который Doxygen может использовать для генерации документации.

```
sudo dnf install pip3
pip3 install doxyqml
```

2.13.2 Настройка Doxygen

Чтобы настроить Doxygen на использование Doxyqml, необходимо внести несколько изменений в файл конфигурации Doxygen.

Добавьте расширение .qml к ключу FILTER_PATTERNS:

```
FILTER_PATTERNS = *.qml=doxyqml
```

Примечание: в Windows Doxyqml устанавливается в папку Scripts вашей установки Python. Если эта папка не находится в PATH, добавьте её или используйте полный путь к Doxyqml (но это менее удобно для разных компьютеров)

Добавьте расширение .qml к FILE_PATTERNS:

```
FILE_PATTERNS = *.qml
```

Начиная с Doxygen 1.8.8, вы также должны добавить расширение .qml в EXTENSION_MAPPING:

```
EXTENSION_MAPPING = qml=C++
```

2.13.3 Документирование типов. Функции

QML частично типизирован: функции нетипизированы, а свойства и сигналы типизированы. Doxyqml позволяет определять типы, если они отсутствуют или недостаточно точны.

Типы в документации можно определить следующим образом:

```
/*!  
 * Create a user  
 * \param type:string firstname User firstname  
 * \param type:string lastname User lastname  
 * \param type:int User age  
 * \return type:User The User object  
 */  
function createUser(firstname, lastname, age);
```

2.13.4 Свойства

Свойства QML типизированы, поэтому Doxyqml использует их по умолчанию. Тем не менее вы можете переопределить тип, используя тот же синтаксис type:<name>. Это полезно для документирования псевдонимов свойств:

```
/*! type:string The user lastname */  
property alias lastname: someObject.text
```

2.13.5 Сигналы

Сигналы QML типизированы, поэтому нет необходимости использовать синтаксис type:<name> для документирования их параметров. Использование синтаксиса type:<name> в документации сигналов не работает: Doxyqml не удалит его, а Doxygen перепутает его с именем параметра.

```
/**
 * User just logged in
 * @param user The user which logged in
 */
signal loggedIn(User user)
```

2.13.6 Извлечение внутренних элементов

Элементы QML с идентификатором экспортируются как приватные переменные-члены.

Для их отображения требуется включить:

```
EXTRACT_ALL = YES
EXTRACT_PRIVATE = YES
```

2.14 Файл Doxyfile и базовые настройки

Отредактируйте Doxyfile, установив следующие параметры:

```
# Имя проекта
PROJECT_NAME = "Мой проект"
# Версия проекта
PROJECT_NUMBER = 1.0
# Краткое описание проекта
PROJECT_BRIEF = "Описание проекта на русском языке"
# Кодировка исходного кода (обязательно для русского языка)
DOXYFILE_ENCODING = UTF-8
# Язык документации
OUTPUT_LANGUAGE = Russian
# Директория для вывода документации
OUTPUT_DIRECTORY = docs
# Формат вывода (HTML, для теста)
GENERATE_HTML = YES
# Включить генерацию LaTeX
GENERATE_LATEX = YES
# Где искать исходные файлы
```

```
INPUT = src include
# Шаблоны файлов для обработки
FILE_PATTERNS = *.cpp *.h *.hpp *.c *.cc
# Рекурсивный обход директорий
RECURSIVE = YES
# Извлекать все элементы (даже недокументированные) E
XTRACT_ALL = YES
# Включать приватные члены класса
EXTRACT_PRIVATE = YES
# Включать статические члены
EXTRACT_STATIC = YES
# Генерировать диаграммы классов (требуется Graphviz)
HAVE_DOT = YES
# Отображать иерархию классов
CLASS_DIAGRAMS = YES
# Отображать графы зависимостей
COLLABORATION_DIAGRAMS = YES
# Краткие описания в стиле JavaDoc
JAVADOC_AUTOBRIEF = YES
# Поддерживать Markdown в комментариях
MARKDOWN_SUPPORT = YES
# Включить поиск в HTML-документации
SEARCHENGINE = YES
# Показать дерево файлов в навигации
TREEVIEW = YES
# Сортировка членов класса по алфавиту
SORT_MEMBER_DOCS = YES
# Количество столбцов в HTML-таблицах
HTML_EXTRA_STYLESHEET = ""
# Заголовок HTML-страницы
HTML_ALIGN_MEMBERS = YES
# Отображение даты генерации
HTML_TIMESTAMP = YES
# Цветовая схема (можно выбрать другую)
HTML_COLORSTYLE_HUE = 220
HTML_COLORSTYLE_SAT = 100
HTML_COLORSTYLE_GAMMA = 80
```

```
# Игнорировать определённые директории
EXCLUDE_PATTERNS = */build/* */test/* */tests/* */vendor/*
# Логотип проекта
HTML_EXTRA_FILES = logo.png
# Поддержка UTF-8 в LaTeX (если GENERATE_LATEX = YES)
LATEX_OUTPUT_ENCODING = UTF-8
# Кодировка файла конфигурации
DOXYFILE_ENCODING = UTF-8
# Язык вывода
OUTPUT_LANGUAGE = Russian
# Кодировка HTML-вывода
HTML_FILE_ENCODING = UTF-8
# Для LaTeX (если используется)
LATEX_OUTPUT_ENCODING = UTF-8
```

2.15 Минимизация документации

Чтобы запретить вывод исходных текстов в документации Doxygen, нужно настроить параметры в файле конфигурации Doxyfile.

```
SHOW_FILES = NO
```

Отключает отображение списка файлов в навигационной панели и ссылки на просмотр исходного кода.

```
SHOW_FILES = NO
VERBATIM_HEADERS = NO
```

Запрещает отображение содержимого заголовочных файлов (.h) в документации.

```
VERBATIM_HEADERS = NO
SOURCE_BROWSER = NO
```

Полностью отключает браузер исходного кода — функцию просмотра файлов с подсветкой синтаксиса. Это наиболее важный параметр для скрытия исходного кода.

```
SOURCE_BROWSER = NO
INLINE_SOURCES = NO
```

Не встраивает исходный код функций/методов прямо в документацию рядом с их описанием.

```
INLINE_SOURCES = NO
SHOW_USED_FILES = NO
```

Убирает блок в конце страниц вида «Документация для этого класса была сгенерирована из файла...».

```
SHOW_USED_FILES = NO
```

Удаляет комментарии из отображаемого исходного кода (если он всё же выводится). Полезно как дополнительная мера.

```
STRIP_CODE_COMMENTS = YES
EXTRACT_ALL = NO #(если установлено YES)
```

Если эта опция включена, Doxygen документирует все сущности, даже без комментариев. Отключение уменьшит объём автоматически генерируемой информации.

```
EXTRACT_ALL = NO
HIDE_UNDOC_MEMBERS = YES
HIDE_UNDOC_CLASSES = YES
```

Скрывают недокументированные члены и классы, уменьшая количество ссылок на файлы.

```
HIDE_UNDOC_MEMBERS = YES
HIDE_UNDOC_CLASSES = YES
```


Источники

Страницы документации

1. Документирование кода: <https://www.doxygen.nl/manual/docblocks.html>
2. Настройка выходных данных:
<https://www.doxygen.nl/manual/customize.html>
3. Дополнительная документация:
<https://www.doxygen.nl/manual/additional.html>
4. Выходные форматы: <https://www.doxygen.nl/manual/output.html>
5. Автоматическая генерация ссылок:
<https://www.doxygen.nl/manual/autolink.html>
6. Поддержка Markdown: <https://www.doxygen.nl/manual/markdown.html>
7. HTML — команды: <https://www.doxygen.nl/manual/htmlcmds.html>

ПРИЛОЖЕНИЕ А

Одностраничная документация в формате MD

Пример оформления руководства пользователя в одном документе.

```
# Пользовательская документация для проекта «База данных метеоритов»  
(Doxygen)
```

```
## Обзор проекта
```

```
**Название:** База данных метеоритов (MeteoriteDB)
```

```
**Тип:** настольное приложение на Qt6
```

```
**Назначение:** учёт и визуализация данных о метеоритах с возможностью  
редактирования, фильтрации и экспорта.
```

```
**Ключевые возможности:**
```

- * управление записями о метеоритах (добавление, редактирование, удаление);
- * визуализация данных в табличном виде и на графиках;
- * импорт данных из CSV/JSON;
- * экспорт данных в CSV;
- * гибкая настройка базы данных и логов;
- * логирование операций и ошибок.

```
## Структура проекта
```

```
...
```

```
project/  
├── CMakeLists.txt  
├── src/  
│   ├── main.cpp  
│   ├── MainWindow.cpp  
│   ├── TabInfo.cpp  
│   ├── TabEdit.cpp  
│   ├── TabTable.cpp  
│   ├── TabGraphs.cpp  
│   ├── TabSettings.cpp  
│   ├── DBManager.cpp  
│   ├── FileManager.cpp  
│   └── Logger.cpp
```

```

├── include/
│   ├── MainWindow.h
│   ├── TabInfo.h
│   ├── TabEdit.h
│   ├── TabTable.h
│   ├── TabGraphs.h
│   ├── TabSettings.h
│   ├── DBManager.h
│   ├── FileManager.h
│   └── Logger.h
├── resources/
│   ├── database.db
│   ├── settings.ini
│   ├── log.txt
│   ├── icons/
│   └── styles/
└── build/
...

```

Описание классов

`MainWindow`

Файл: `include/MainWindow.h`

Родитель: `QMainWindow`

Описание: главное окно приложения с вкладками. Сохраняет и загружает позицию и размер окна через `QSettings`.

Методы:

* `MainWindow(QWidget \*parent = nullptr)` – конструктор, инициализирует компоненты;

* `~MainWindow()` – деструктор, сохраняет настройки перед закрытием;

* `setupUI()` – создаёт и настраивает интерфейс (вкладки, макет);

* `loadSettings()` – загружает настройки окна (позиция, размер) из `settings.ini`;

* `saveSettings()` – сохраняет настройки окна в `settings.ini`.

Связи: использует `DBManager`, `Logger` и все вкладки (`TabInfo`, `TabEdit` и т.д.).

`DBManager`

Файл: `include/DBManager.h`

Родитель: `QObject`

Описание: класс для работы с SQLite. Обеспечивает подключение к БД, создание таблиц, выполнение CRUD-операций и обработку ошибок.

Основные методы:

```

* `bool connectToDatabase(const QString &databasePath)` – подключает к БД
по указанному пути;
* `bool createTables()` – создаёт таблицу `meteorites`, если её нет;
* `bool deleteDatabase()` – удаляет файл БД;
* `bool isTableExists() const` – проверяет существование таблицы
`meteorites`;
* ` QSqlDatabase database() const` – возвращает объект подключения к БД.

**Таблица `meteorites`:**
* `id` (`INTEGER`, PRIMARY KEY, AUTOINCREMENT) – уникальный идентификатор;
* `name` (`TEXT`) – название метеорита;
* `location` (`TEXT`) – место падения;
* `type` (`TEXT`) – тип метеорита;
* `mass` (`REAL`) – масса в кг;
* `discovery_year` (`INTEGER`) – год обнаружения;
* `chemical_composition` (`TEXT`) – химический состав;
* `scientific_value` (`TEXT`) – научное значение.

### `Logger`
**Файл:** `include/Logger.h`
**Родитель:** `QObject`
**Описание:** класс логирования. Записывает сообщения с временными
метками, очищает и удаляет лог-файл.

**Основные методы:**
* `void log(const QString &message, const QString &level = "INFO")` –
записывает сообщение в лог;
* `bool clearLog()` – очищает содержимое лог-файла;
* `bool deleteLog()` – удаляет лог-файл;
* `qint64 getLogSize() const` – возвращает размер лога в байтах;
* `QDateTime getLastModified() const` – возвращает дату последнего
изменения лога.

**Формат лога:**
...
[LEVEL] yyyy-MM-dd hh:mm:ss: сообщение
...

Пример:
...
[ERROR]      2024-05-20      14:30:15:      Ошибка      подключения      к      БД:
resources/database.db
...

### `TabSettings`
**Файл:** `include/TabSettings.h`
**Родитель:** `QWidget`

```

****Описание:**** вкладка настроек приложения. Управляет путём к БД, созданием/удалением БД и таблиц, мониторингом и управлением логами.

****Элементы интерфейса:****

- * поле ввода пути к БД (``QLineEdit``);
- * кнопка «Обзор» (``QPushButton``) – открывает диалог выбора файла;
- * кнопки «Создать БД и таблицу» и «Удалить БД и таблицу»;
- * информация о логге (``QLabel``) – размер и дата изменения;
- * кнопки «Очистить лог» и «Удалить лог»;
- * статус-индикаторы (``QLabel``): «Статус БД» (подключено/не подключено), «Статус логга» (доступен/недоступен);
- * кнопка «Сохранить настройки».

****Основные методы:****

- * ``onBrowseDatabasePath()`` – открывает диалог выбора файла БД;
- * ``onCreateDatabase()`` – создаёт БД и таблицу ``meteorites``;
- * ``onDeleteDatabase()`` – запрашивает подтверждение и удаляет БД;
- * ``onClearLog()`` – очищает лог;
- * ``onDeleteLog()`` – удаляет лог;
- * ``onSaveSettings()`` – сохраняет путь к БД в ``settings.ini``;
- * ``updateStatusIndicators()`` – обновляет статус-индикаторы и информацию о логге.

Другие вкладки

- * ****TabInfo**** (``TabInfo.h/cpp``) – отображает и редактирует информацию о студенте (ФИО, группа, программа) из ``settings.ini``.

- * ****TabEdit**** (``TabEdit.h/cpp``) – форма ввода/редактирования записей о метеоритах. Взаимодействует с ``DBManager`` для сохранения данных. Поддерживает drag-and-drop для импорта CSV/JSON.

- * ****TabTable**** (``TabTable.h/cpp``) – таблица (``QTableView``) с данными из БД. Реализует фильтрацию, сортировку и экспорт в CSV. Привязана к ``QSqlTableModel``.

- * ****TabGraphs**** (``TabGraphs.h/cpp``) – визуализирует данные: карта мест падений и гистограмма распределения по массе. Позволяет сохранять диаграммы в изображения.

Использование приложения

Запуск

1. Соберите проект с помощью CMake:

```
```bash
mkdir build && cd build
cmake ..
make
./MeteoriteDB
```
```

2. Приложение запустится с главным окном и вкладкой «Информация».

Вкладки

* ****Информация:**** заполните и сохраните данные о студенте в `settings.ini`. Нажмите «О программе» для просмотра описания функционала.

* ****Редактирование:**** заполните поля формы и используйте кнопки:

* «Добавить» — добавляет новую запись в БД;

* «Сохранить» — обновляет существующую запись;

* «Удалить» — удаляет выбранную запись;

* «Очистить форму» — сбрасывает поля ввода.

Поддерживается drag-and-drop CSV/JSON для импорта данных.

* ****Таблица данных:**** отображает содержимое БД в виде таблицы. Используйте поле поиска над таблицей для фильтрации по столбцам. Сортируйте данные кликом на заголовок столбца. Нажмите кнопку «Экспорт в CSV» для сохранения данных.

* ****Диаграммы:**** визуализирует данные на карте и гистограмме. Нажмите «Обновить данные» для пересчёта графиков. Сохраните диаграмму в изображение кнопкой «Сохранить как изображение».

* ****Настройки:****

* установите путь к БД через поле ввода или кнопку «Обзор»;

* создайте БД кнопкой «Создать БД и таблицу»;

* удалите БД кнопкой «Удалить БД и таблицу» (с подтверждением);

* очистите или удалите лог кнопками «Очистить лог»/«Удалить лог»;

* сохраните настройки кнопкой «Сохранить настройки»;

* отслеживайте статус подключения к БД и доступности лога по индикаторам.

Настройки и логи

* ****Файл настроек:**** `resources/settings.ini` (хранит путь к БД, позицию окна и т.д.).

* ****База данных:**** `resources/database.db` (SQLite).

* ****Лог-файл:**** `resources/log.txt` (текстовый файл с записями операций и ошибок).

Сборка проекта

****Требования:****

* Qt6 (Core, Widgets, Sql, Graphs);

* CMake ≥ 3.22;

* компилятор с поддержкой C++20.

****Шаги сборки:****

1. ****Клонирование репозитория:****

```
```bash
```

```
git clone <url-репозитория>
```

```
cd MeteoriteDB
```

...

2. **\*\*Создание директории сборки:\*\***

```
```bash
mkdir build
cd build
```
```

3. **\*\*Конфигурация проекта через CMake:\*\***

```
```bash
cmake ..
```
```

Или с указанием компилятора:

```
```bash
cmake -DCMAKE_CXX_COMPILER=g++ ..
```
```

4. **\*\*Компиляция проекта:\*\***

```
```bash
make
```
```

Для ускорения сборки можно использовать параллельную компиляцию:

```
```bash
make -j$(nproc)
```
```

5. **\*\*Запуск приложения:\*\***

```
```bash
./MeteoriteDB
```
```

**\*\*Возможные проблемы при сборке и их решения:\*\***

\* **\*\*Ошибка: «Qt6 не найден»\*\*** – установите Qt6 через пакетный менеджер:

- \* Ubuntu/Debian: ``sudo apt install qt6-base-dev qt6-tools-dev``
- \* Fedora: ``sudo dnf install qt6-qtbase-devel qt6-qtsvg-devel``
- \* Windows: установите Qt через Qt Online Installer.

\* **\*\*Ошибка компиляции C++20\*\*** – убедитесь, что компилятор поддерживает C++20:

- \* GCC  $\geq 10$ ;
- \* Clang  $\geq 12$ ;
- \* MSVC  $\geq 2019$  (с флагом ``/std:c++20``).

\* **\*\*Отсутствие библиотек Qt6::Graphs\*\*** – установите компонент Graphs:

- \* В Qt Online Installer выберите модуль ``Qt Graphs``.

---

```

Конфигурация и настройки

Файл `settings.ini`

Расположение: `resources/settings.ini`.
Формат: INI (ключ=значение).

Основные секции и параметры:

* **[database]**
 * `path` – путь к файлу БД (по умолчанию: `resources/database.db`).
* **[window]**
 * `position` – позиция главного окна (формат: `x,y`);
 * `size` – размер главного окна (формат: `width,height`).
* **[student]**
 * `name` – ФИО студента;
 * `group` – номер группы;
 * `program` – название программы;
 * `version` – версия приложения.

Пример содержимого:
```ini
[database]
path=resources/database.db

[window]
position=100,100
size=1000,700

[student]
name=Иванов Иван Иванович
group=M-123
program=Прикладная информатика
version=1.0.0
```

Файл базы данных

Расположение: `resources/database.db`.
Тип: SQLite.
Таблица: `meteorites` (создаётся автоматически при первом запуске).

Файл логов

Расположение: `resources/log.txt`.

```



**\*\*Формат записей:\*\***

...

[LEVEL] yyyy-MM-dd hh:mm:ss: сообщение

...

**\*\*Уровни логирования:\*\***

\* `INFO` – информационные сообщения;

\* `WARNING` – предупреждения;

\* `ERROR` – ошибки.

---

**## Руководство пользователя**

**### Начальный запуск**

1. Запустите приложение `MeteoriteDB`.
2. Перейдите на вкладку **\*\*«Настройки»\*\***:
  - \* проверьте путь к БД (по умолчанию: `resources/database.db`);
  - \* нажмите **\*\*«Создать БД и таблицу»\*\***, если файл БД отсутствует;
  - \* сохраните настройки кнопкой **\*\*«Сохранить настройки»\*\***.
3. Перейдите на вкладку **\*\*«Информация»\*\*** и заполните данные о студенте.

**### Работа с данными**

**\*\*Добавление записи:\*\***

1. Перейдите на вкладку **\*\*«Редактирование»\*\***.
2. Заполните поля формы:
  - \* **\*\*name\*\*** – название метеорита;
  - \* **\*\*location\*\*** – место падения (координаты или описание);
  - \* **\*\*type\*\*** – тип (каменный, железный и т.д.);
  - \* **\*\*mass\*\*** – масса в кг;
  - \* **\*\*discovery\_year\*\*** – год обнаружения;
  - \* **\*\*chemical\_composition\*\*** – химический состав;
  - \* **\*\*scientific\_value\*\*** – научное значение.
3. Нажмите кнопку **\*\*«Добавить»\*\***.

**\*\*Редактирование записи:\*\***

1. Выберите запись в таблице на вкладке **\*\*«Таблица данных»\*\***.
2. Данные автоматически отобразятся в форме на вкладке **\*\*«Редактирование»\*\***.
3. Внесите изменения и нажмите **\*\*«Сохранить»\*\***.

**\*\*Удаление записи:\*\***

1. Выберите запись в таблице или форме.
2. Нажмите **\*\*«Удалить»\*\*** на вкладке **\*\*«Редактирование»\*\***.

**\*\*Импорт данных:\*\***

1. Перетащите CSV/JSON-файл в область формы на вкладке **\*\*«Редактирование»\*\***.
2. Данные будут автоматически добавлены в БД.

**### Визуализация данных**

**\*\*На вкладке «Диаграммы»:\*\***

- \* **\*\*Карта мест падений\*\*** – отображает географическое распределение метеоритов.
- \* **\*\*Гистограмма массы\*\*** – показывает распределение метеоритов по массе.
- \* Нажмите **\*\*«Обновить данные»\*\*** для пересчёта графиков.
- \* Сохраните диаграмму в изображение кнопкой **\*\*«Сохранить как изображение»\*\***.

**### Экспорт данных**

**\*\*Экспорт в CSV:\*\***

1. На вкладке **\*\*«Таблица данных»\*\*** нажмите кнопку **\*\*«Экспорт в CSV»\*\***.
2. Выберите путь и имя файла.
3. Данные из таблицы будут сохранены в формате CSV.

**### Управление настройками и логами**

**\*\*Изменение пути к БД:\*\***

1. В поле **\*\*«Путь к БД»\*\*** на вкладке **\*\*«Настройки»\*\*** введите новый путь или выберите файл кнопкой **\*\*«Обзор»\*\***.
2. Нажмите **\*\*«Сохранить настройки»\*\***.

**\*\*Очистка логов:\*\***

1. Нажмите **\*\*«Очистить лог»\*\*** на вкладке **\*\*«Настройки»\*\***.
2. Содержимое `log.txt` будет удалено, сам файл сохранится.

**\*\*Удаление логов:\*\***

1. Нажмите **\*\*«Удалить лог»\*\***.
2. Подтвердите действие в диалоговом окне.
3. Файл `log.txt` будет удалён с диска.

---

**## Обработка ошибок**

**\*\*Типичные ошибки и решения:\*\***

- \* **\*\*«Не удалось подключиться к БД»\*\***:
  - \* проверьте существование файла БД;

- \* убедитесь, что путь к БД указан верно;
- \* создайте БД кнопкой **«Создать БД и таблицу»**.
- \* **«Ошибка импорта CSV/JSON»**:
  - \* проверьте формат файла (должен соответствовать структуре таблицы `meteorites`);
  - \* убедитесь, что файл не повреждён.
- \* **«Нет прав доступа к файлу»**:
  - \* запустите приложение с правами администратора (если требуется);
  - \* измените права доступа к файлам в директории `resources/`.
- \* **«Ошибки логирования»**:
  - \* проверьте, что файл `log.txt` не заблокирован другим процессом;
  - \* очистите или удалите лог через вкладку **«Настройки»**.

---

## ## Дополнительные рекомендации

- \* **Резервное копирование БД**: перед массовыми операциями (импорт, удаление) скопируйте файл `database.db` вручную.
- \* **Оптимизация производительности**: для больших объёмов данных используйте индексы в SQLite (можно добавить в `DBManager::createTables()`).
- \* **Расширение функционала**:
  - \* добавьте поддержку других форматов экспорта (JSON, Excel);
  - \* реализуйте поиск по всей БД на вкладке **«Таблица данных»**;
  - \* интегрируйте онлайн-карты (например, OpenStreetMap) для визуализации мест падений.

---

## ## Контактная информация

**Автор проекта**: [Ваше имя]  
**Email**: [ваш email]  
**Репозиторий**: [ссылка на GitHub/GitLab]

Если у вас возникли вопросы или предложения по улучшению проекта, пожалуйста, свяжитесь с автором.